VEGOVA
ELEKTROTEHNIŠKO-RAČUNALNIŠKA
STROKOVNA ŠOLA IN GIMNAZIJA
LJUBLJANA

Splošna matura
Seminarska naloga pri predmetu računalništvo

Aplikacija za sledenje javnemu prometu

Mentor: Matjaž Majnik, univ. dipl. inž.

Avtor: Jaka Černetič, G 4. A

Ljubljana, april 2026

Povzetek

Seminarska naloga opisuje razvoj progresivne spletne aplikacije IJPP Tracker, namenjene sprotnemu sledenju vozil javnega potniškega prometa v Republiki Sloveniji. Aplikacija združuje podatke treh neodvisnih sistemov – Ljubljanskega potniškega prometa (LPP), Integriranega javnega potniškega prometa (IJPP) in Slovenskih železnic (SŽ) – ter jih prikazuje na interaktivnem zemljevidu. Naloga obravnava arhitekturo sistema, uporabljene tehnologije in ključne izzive, ki so se pojavili med razvojem: neujemanje identifikatorjev postaj med podsistemi, heterogenost podatkovnih vmesnikov ter potrebo po dinamičnem grajenju zahtevnih spletnih naslovov.

Ključne besede: javni potniški promet, React, PWA, MapLibre GL, REST API, GTFS, LPP, IJPP, SŽ

Abstract

This paper describes the development of a progressive web application (PWA) called IJPP Tracker, designed for real-time tracking of public transport vehicles in the Republic of Slovenia. The application aggregates data from three independent systems — Ljubljana City Bus (LPP), Integrated Public Transport (IJPP), and Slovenian Railways (SŽ) — and displays them on an interactive map. The paper covers the system architecture, the technologies used, and the key challenges encountered during development: mismatched station identifiers across subsystems, heterogeneous data interfaces, and the need for dynamically constructed API request URLs.

Keywords: public transport, React, PWA, MapLibre GL, REST API, GTFS, LPP, IJPP, SŽ

Kazalo

1 Uvod **5**

2 Pregled področja **5**

2.1 Javni potniški promet v digitalnem okolju 5

2.2 Obstoječe rešitve in njihove pomanjkljivosti 6

2.3 Tehnični standardi in protokoli 6

3 Arhitektura aplikacije **6**

3.1 Pregled arhitekturne zasnove 6

3.2 Odjemalska stran - React SPA 7

3.3 Strežniška plast – brezstrežniške funkcije (serverless functions) 7

3.4 Upravljanje stanja aplikacije 7

4 Podatkovni sloj - modul `Api.jsx` **8**

4.1 Pregled modula 8

4.2 Mehanizem predpomnjenja 8

4.3 Normalizacija prihodov 8

4.4 Pridobivanje položajev vozil 9

4.5 Sledenje vlakom z interpoliranjem položaja 10

5 Kartografski prikaz **10**

5.1 Knjižnica MapLibre GL 10

5.2 Organizacija slojev karte 11

5.3 Gruče postajališč in avtobusov 11

5.4 Prikaz izbrane poti 12

5.5 Prilagodljiva frekvenca osveževanja 12

6 Uporabniški vmesnik **13**

6.1 Zavihek Postaje 13

6.2 Zavihek Linije 13

6.3 Zavihek Nastavitve 14

7 Izzivi med razvojem in rešitve **14**

7.1 Neujemanje postajališč med podsistemi 14

7.2 Heterogenost podatkovnih vmesnikov 15

7.3 Dinamično posodabljanje URL-ja za pridobivanje položajev vlakov 15

7.4 Dekodiranje poti vlakov 16

8 Namestitev in gostovanje **17**

8.1 Orodje za gradnjo Vite 17

8.2 Progresivna spletna aplikacija 17

8.3 Nalaganje aplikacije na GitHub Pages in Vercel 17

9 Zaključek **17**

10 Priloge **18**

10.1 Izvorna koda aplikacije 18

11 Viri in literatura **19**

1 Uvod

Javni potniški promet v Republiki Sloveniji deluje v okviru večjih, med seboj delno ločenih sistemov. V Mestni občini Ljubljana avtobusni promet zagotavlja Javno podjetje Ljubljanski potniški promet d.o.o. (LPP), medregijski in primestni avtobusni promet pa koordinira Ministrstvo za infrastrukturo v okviru sistema Integrirani javni potniški promet (IJPP), ki vključuje prevoznike, kot so Arriva Slovenija, Nomago in Marprom. Železniški promet operira v okviru družbe Slovenske železnice d.o.o. (SŽ).

Kljub temu da vsak od omenjenih sistemov zagotavlja določeno stopnjo digitalnih informacij za potnike, ti sistemi med seboj niso integrirani v enotno uporabniško izkušnjo. Uporabnik, ki želi preveriti, kdaj bo prispel naslednji mestni ali medkrajevni avtobus in vlak na skupni postajališčni točki, mora obiskati vsaj tri različne spletne ali mobilne aplikacije. Ta razdrobljenost je bila ena izmed osnovnih motivacij za razvoj aplikacije IJPP Tracker.

Namen seminarske naloge je celovita predstavitev razvoja progresivne spletne aplikacije (Progressive Web App - PWA), ki združuje podatkovne tokove vseh treh navedenih prometnih podsistemov v enoten vmesnik. Aplikacija omogoča prikaz trenutnih lokacij vozil na interaktivnem zemljevidu, iskanje postajališč, pregledovanje prihodov in sledenje posameznim linijam.

V nalogi so postavljene naslednje hipoteze: (1) sodobne spletne tehnologije omogočajo vzpostavitev zmogljive aplikacije za sledenje javnega potniškega prometa brez nativnega mobilnega razvoja; (2) razlike med podatkovnimi vmesniki posameznih sistemov je mogoče premostiti z ustrezno abstrakcijsko plastjo na strani odjemalca; (3) omejitve, ki jih prinesejo varnostne politike brskalnikov (CORS - Cross-Origin Resource Sharing), zahtevajo posredovalni strežniški sloj, ki lahko učinkovito posreduje zahteve do zunanjih API-jev.

2 Pregled področja

2.1 Javni potniški promet v digitalnem okolju

Digitalizacija javnega potniškega prometa poteka na dveh ravneh. Na ravni zbiranja podatkov gre za nameščanje GPS-sprejemnikov in senzorjev v vozila, ki oddajajo podatke o položaju vozil v osrednji sistem. Na ravni distribucije gre za objavo teh podatkov prek standardiziranih vmesnikov, ki jih najenostavneje razumemo kot spletne naslove, na katere se poveže zunanji odjemalec in pridobi strukturirane podatke.

Mednarodni standard za izmenjavo podatkov o javnem potniškem prometu je GTFS (General Transit Feed Specification), ki ga je prvotno razvil Google v sodelovanju s Portlandsko metropolitansko agencijo za transport [2]. Standard GTFS definira statične podatke (vozni redi, postajališča, linije), medtem ko razširitev GTFS-Realtime omogoča prenos dinamičnih podatkov (zamude, pozicije vozil, opozorila). Čeprav slovensko potniško omrežje v večji meri sledi tem standardom, posamezni sistemi implementirajo lastne razširitve in nestandardne vmesnike.

2.2 Obstoječe rešitve in njihove pomanjkljivosti

Na slovenskem trgu obstaja več delnih rešitev. Uradna aplikacija LPP Ljubljana [3] pokriva izključno mestne avtobuse v Ljubljani. Spletni portal BrezAvta [9] nudi načrtovanje poti od točke do točke za medkrajevne avtobuse, ni pa primeren za spremljanje trenutnih pozicij avtobusov. Portal SŽ [8] omogoča vpogled v red vožnje, ne prikazuje pa položajev vlakov na zemljevidu.

Skupna pomanjkljivost vseh navedenih rešitev je odsotnost enotnega pogleda, ki bi na enem mestu prikazoval vse oblike javnega prometa s skupnimi funkcijami iskanja in sledenja. Prav to vrzel zapolnjuje aplikacija IJPP Tracker.

2.3 Tehnični standardi in protokoli

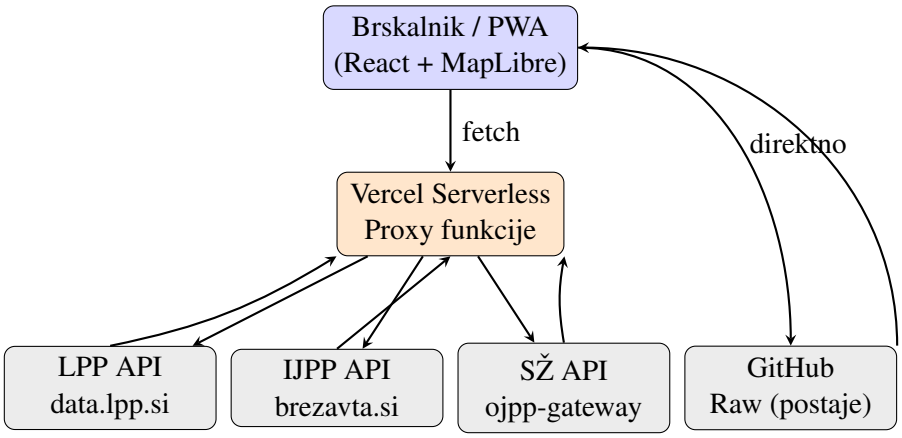
Za razumevanje arhitekture aplikacije je ključno poznavanje dveh tehničnih konceptov. Prvi je REST API (Representational State Transfer Application Programming Interface), ki je arhitekturni slog za gradnjo spletnih vmesnikov, pri katerem odjemalec pošlje zahtevo HTTP na določen URL in prejme podatke v obliki JSON. Drugi je mehanizem CORS (Cross-Origin Resource Sharing), varnostna politika brskalnikov, ki preprečuje, da bi skripta na enem spletnem naslovu neposredno dostopala do podatkov na drugem spletnem naslovu brez ustreznega dovoljenja strežnika [6]. Ker nekateri javni API-ji ne nastavljajo ustreznih glav CORS, aplikacija potrebuje posredovalni strežnik (proxy), ki zahteve posreduje v imenu odjemalca.

3 Arhitektura aplikacije

3.1 Pregled arhitekturne zasnove

Aplikacija IJPP Tracker je zgrajena kot enostranska aplikacija (Single Page Application – SPA) z dvonivojsko arhitekturo. Odjemalska stran je progresivna spletna aplikacija, razvita z ogrodjem React, ki teče neposredno v brskalniku. Strežniška stran je sestavljena iz brezstrežniških funkcij (serverless functions), ki delujejo na platformi Vercel in služijo izključno kot CORS-posredniki.

Slika 1 prikazuje visokonivojski pogled na arhitekturo.



Slika 1: Visokonivojska arhitektura aplikacije IJPP Tracker

Ločevanje odjemalske in strežniške plasti zagotavlja, da je vsa logika prikaza in obdelave

podatkov na strani odjemalca, medtem ko strežniška plast služi zgolj kot posrednik pri zahtevah, ki jih brskalniki sicer blokirajo zaradi varnostnih politik CORS. Statični podatki, kot so seznami postaj in poti linij, so shranjeni na platformi GitHub v obliki JSON datotek in jih aplikacija pridobi neposredno brez proxy posrednika, saj ta endpoint dovoljuje CORS dostop.

3.2 Odjemalska stran - React SPA

Odjemalska aplikacija je organizirana v hierarhijo komponent, pri čemer je korenski element komponenta `App`, ki upravlja globalno stanje in koordinira izmenjavo podatkov med zavihki. Navigacijo med štirimi glavnimi razdelki (zemljevid, postaje, linije, nastavitve) zagotavlja knjižnica React Router z načinom hash routinga, ki je primeren za statično gostovanje na platformah brez podpore za strežniško usmerjanje.

Komponente zavihkov so naložene z leno inicializacijo (`React.lazy`), kar pomeni, da se koda posameznega zavihka prenese šele ob prvem obisku tega zavihka. S tem se skrajša čas do prve interaktivnosti aplikacije.

3.3 Strežniška plast – brezstrežniške funkcije (serverless functions)

Mapa `api/` vsebuje šest JavaScript datotek, ki se ob namestitvi na Vercel samodejno pretvorijo v HTTP-končne točke. Vsaka datoteka sledi vzorcu CORS-posrednika:

- sprejme zahtevo od odjemalca,
- jo posreduje ciljnemu API-ju,
- doda ustrezne glave `Access-Control-Allow-Origin`,
- vrne odgovor odjemalcu.

Funkcije so implementirane v dveh izvajalnih okoljih (runtime).

Nekatere, na primer `lpp-arrivals.js`, so konfigurirane kot `runtime: "nodejs"`. To je potrebno, ker vsebujejo logiko za ponovitev zahteve ob napaki in predpomnjenje. Druge, na primer `lpp-stops.js`, pa so konfigurirane kot `runtime: "edge"`, kar pomeni izvajanje v globalni mreži robnih vozlišč z nižjo latenco, a brez dostopa do polnega Node.js API-ja.

3.4 Upravljanje stanja aplikacije

Globalno stanje aplikacije je centralizirano v komponenti `App` in zajema: seznam postajališč (`busStops`, `szStops`), trenutne položaje vozil (`gpsPositions`, `trainPositions`), aktivno postajališče (`activeStation`), prihode za aktivno postajališče (`lppArrivals`, `ijppArrivals`, `szArrivals`), izbrano vozilo (`selectedVehicle`) in nastavitve vidnosti plasti na zemljevidu (`visibility`, `busOperators`).

Trajnost stanja med sejami je zagotovljena s shranjevanjem v `localStorage` brskalnika, kjer se ob vsaki posodobitvi zapišejo nastavitve teme, aktivno postajališče, nastavitve na zemljevidu prikazanih slojev in seznam priljubljenih postajališč.

4 Podatkovni sloj - modul `Api.jsx`

4.1 Pregled modula

Datoteka `src/Api.jsx` je osrednji podatkovni sloj aplikacije in obsega okoli 700 vrstic kode. Vsebuje vse funkcije za pridobivanje podatkov iz zunanjih virov, logiko za predpomnjenje, pomožne funkcije za formatiranje časovnih podatkov ter algoritme za dekodiranje geografskih poti in interpolacijo položajev vozil.

4.2 Mehanizem predpomnjenja

Aplikacija implementira lastno dvoplastno shemo predpomnjenja, ki sledi različnim veljavnostim podatkov glede na njihovo naravo:

Tabela 1: Veljavnost predpomnjenih podatkov glede na tip

Tip podatkov	Veljavnost	Utemeljitev
Postajališča	5 minut	Statični podatki, redko se spreminjajo
Položaji vozil	5 sekund	Sprotni podatki
Prihodi	15 sekund	Posodabljanje napovedi prihodov
Poti (routes)	5 minut	Geometrija poti se ne spreminja med vožnjo

Splošno predpomnjenje zagotavlja objekt `Map` z imenom `cache`, ki za vsak URL ali ključ hrani par (podatki, časovni žig). Ločen objekt `routeCache` skrbi za predpomnjenje geometrij in postajališč posameznih voženj. Objekt `routeInFlight` pa preprečuje podvajanje sočasnih zahtev za isto vožnjo - če zahteva za dani `tripId` že poteka, nova zahteva vrne obstoječo obljubo (Promise), namesto proženja nove HTTP zahteve.

4.3 Normalizacija prihodov

Trije podatkovni viri prihodov vračajo čase v različnih formatah: LPP API vrača koliko minut je še do prihoda (`eta_min`), IJPP API vrača sekunde od polnoči, SŽ API pa vrača ISO 8601 časovne žige. Funkcija `computeEtaAndTime` normalizira vse tri formate v enoten par (minute do prihoda, ura v formatu HH:mm):

```
function computeEtaAndTime(arrival) {
  let etaMin = arrival.etaMinutes ?? arrival.eta_min ?? undefined;
  let actualTime =
    arrival.realtimeDeparture ||
    arrival.actualDeparture ||
    arrival.scheduledDeparture ||
    arrival.estimated_arrival_time ||
    arrival.arrival_time ||
    arrival.realtimeArrival ||
    arrival.scheduledArrival;

  let actualDate = null;
  if (actualTime) {
    actualDate = new Date(actualTime);
```

```
    // If parsing failed or if it looks like a time-only string, try
   prepending today's date
    if (
      isNaN(actualDate.getTime()) &&
      typeof actualTime === "string" &&
      actualTime.match(/^\d{2}:\d{2}(:\d{2})?$/)
    ) {
      const today = new Date().toISOString().split("T")[0];
      actualDate = new Date(`${today}T${actualTime}`);
    }
  }
  if (actualDate && isNaN(actualDate.getTime())) {
    actualDate = null;
  }

  // Generate ETA if missing
  if (etaMin === undefined && actualDate) {
    const now = new Date();
    etaMin = Math.max(
      0,
      Math.round((actualDate.getTime() - now.getTime()) / 60000),
    );
  }

  // Generate actual time if missing
  if (!actualDate && etaMin !== undefined) {
    actualDate = new Date(new Date().getTime() + etaMin * 60000);
  }

  const timeStr = actualDate ? formatTime(actualDate) : "N/A";
  return {
    etaMinutes: etaMin ?? undefined,
    arrivalTime: timeStr,
  };
};
```

Izsek kode 1: Normalizacija ETA iz različnih formatov

Funkcija `seconds2time` pretvori sekunde od polnoči v niz `HH:MM:SS`, kar je potrebno za obdelavo odgovorov IJPP API-ja, ki čase vrača v tej obliki.

4.4 Pridobivanje položajev vozil

Aplikacija sočasno pridobiva položaje mestnih avtobusov LPP in medkrajevnih avtobusov IJPP z vzporednimi klici prek `Promise.all`. Oba seznama se nato združita v enoten niz, ki se posreduje komponentam za prikaz.

LPP API vrača objekte z lastnostmi `latitude`, `longitude`, `line_number` in `trip_id`.

IJPP API vrača objekte v drugačni strukturi z gnezdenimi polji: `vehicle.operator_name`, `trip_headsign` in `stop.name`.

Obe strukturi se v funkcijah `fetchLPPPositions` in `fetchIJPPPositions` prevedeta v enoten format z lastnostmi `gpsLocation`, `operator`, `lineName` in `tripId`.

4.5 Sledenje vlakom z interpoliranjem položaja

Sledenje vlakom je tehnično zahtevnejše kot sledenje avtobusom, saj SŽ API ne oddaja dejanskih položajev vlakov s frekvenco, primerno za gladko gibanje na karti. Rešitev temelji na predhodnem računanju celotne poti vlaka s časovnimi žigi in naknadnim interpoliranju položaja na osnovi sistemskega časa.

Funkcija `fetchTrainPositions` pridobi vse aktivne vlake v oknu 60 sekund. Vsak vlak vsebuje pot v kodirani obliki Polyline in čase odhoda ter prihoda. Iz teh podatkov se zgradi tabela točk z absolutnimi časovnimi žigi:

```
// Izracunaj razdalje med tockami z Haversine formulo
let totalDist = 0;
const dists = [0];
for (let i = 1; i < coords.length; i++) {
  totalDist += haversine(coords[i-1], coords[i]);
  dists.push(totalDist);
}
// Vsaki tocki pripisemo casovni zig glede na delno razdaljo
path = coords.map((coord, i) => ({
  coord,
  time: adjDep + (dists[i] / totalDist) * duration
}));
```

Izsek kode 2: Gradnja časovno označene poti vlaka

Animacijska zanka v komponenti `App` teče s frekvenco 1 Hz z `requestAnimationFrame`.

Ob vsakem ciklu se za vsak vlak pokliče funkcija `getInterpolatedPosition`. Ta interpolira položaj in smer gibanja (bearing):

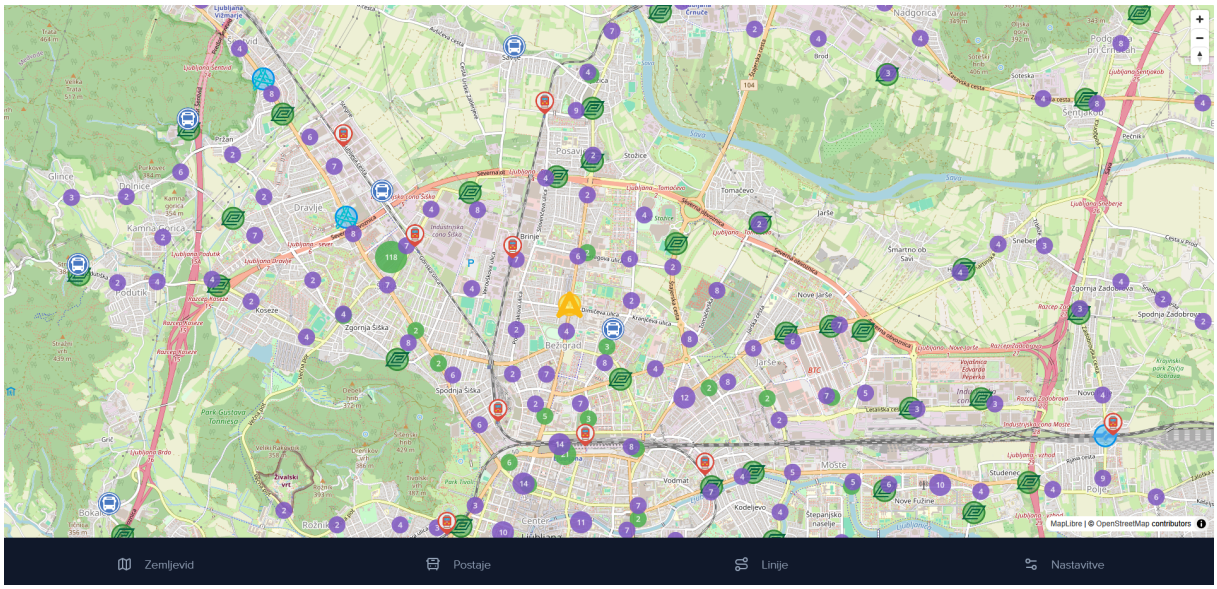
```
for (let i = 1; i < path.length; i++) {
  if (now <= path[i].time) {
    const prev = path[i - 1];
    const curr = path[i];
    const t = (now - prev.time) / (curr.time - prev.time);
    const coord = [
      prev.coord[0] + (curr.coord[0] - prev.coord[0]) * t,
      prev.coord[1] + (curr.coord[1] - prev.coord[1]) * t
    ];
    return { coord, bearing: calcBearing(prev.coord, curr.coord) };
  }
}
```

Izsek kode 3: Interpolacija položaja vlaka na poti

5 Kartografski prikaz

5.1 Knjižnica MapLibre GL

Osnova za kartografski prikaz je knjižnica MapLibre GL JS [4], odprtokodna nadgradnja projekta Mapbox GL JS. Knjižnica za upodabljanje zemljevida uporablja tehnologijo WebGL, kar zagotavlja tekoče gibanje in visoko zmogljivost celo pri prikazu tisočih točk hkrati. Zemljevida sta dva: svetel (OpenStreetMap Raster Light) in temen (OpenStreetMap Raster Dark), med



Slika 2: Zavihek z zemljevidom

njima pa lahko preklopi uporabnik. Na sliki 2 je prikazan svetli zemljevid z izrisanimi položaji avtobusov in postajališč.

5.2 Organizacija slojev karte

Kartografski prikaz je organiziran v module znotraj mape `src/tabs/map/`:

Tabela 2: Moduli kartografskega prikaza

Datoteka	Odgovornost
`config.js`	Konstante, ikone, barvne sheme prevoznikov
`layers.js`	Dodajanje in posodabljanje GeoJSON virov in slojev
`interactions.js`	Pojavna okna (popup) ob kliku na vozilo/postajališče
`popups.js`	HTML predloge za vsebino pojavnih oken
`utils.js`	Pretvorba podatkov v GeoJSON format

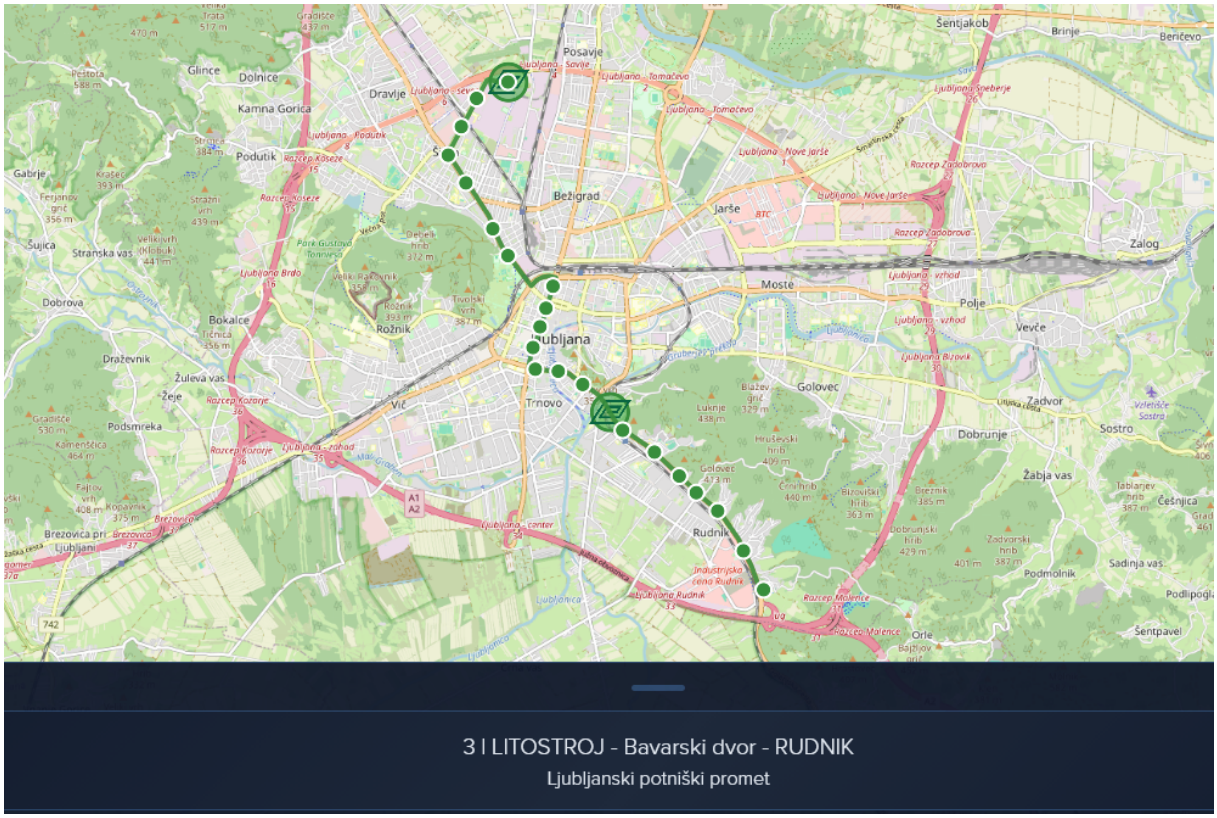
Vsaka entiteta na karti je urejena v plast (*layer*), ki je vezana na vir podatkov (*source*) tipa GeoJSON. Avtobusi, postajališča, vlaki in železniške postaje so ločeni sloji, kar omogoča neodvisno vklapljanje in izklapljanje posameznih slojev zemljevida.

5.3 Gruče postajališč in avtobusov

Postajališča in avtobusi so pri nižji stopnji povečave prikazani v obliki gruč (cluster). Ko je elementov preveč za pregleden prikaz, se samodejno združijo v krog s številko, ki pove, koliko jih je v skupini. Z večanjem povečave se gruče razpustijo v posamezne postaje in avtobuse.

5.4 Prikaz izbrane poti

Ob izbiri vozila iz kateregakoli zavihka se na karti izriše celotna trasa linije s postajišči. Trasa je prikazana kot linijski sloj na vrhu vseh ostalih slojev, postajišča na trasi pa so označena s krogi, ki so barvno usklajeni z barvno shemo prevoznika. Komponenta `RouteTab` prikaže predal s podrobnostmi o vsakem postajališču, vključno s predvidenimi časi prihodov, kot je prikazano na sliki 3.



Slika 3: Prikaz trase izbrane linije na karti

5.5 Prilagodljiva frekvenca osveževanja

Frekvenca pridobivanja podatkov o položajih avtobusov je dinamično prilagojena stopnji povečave karte (zoom level):

Tabela 3: Frekvenca osveževanja glede na stopnjo povečave

Stopnja povečave	Interval	Utemeljitev
Pod 10	15 sekund	Majhna gostota informacij
10–11	10 sekund	Zmerna podrobnost
12–13	5 sekund	Visoka podrobnost
14 in več	2 sekundi	Kar se da sprotno posodabljanje informacij

Dodatno je osveževanje zaustavljeno, kadar je zavihek brskalnika v ozadju (zaznano prek `document.addEventListener("visibilitychange", ...)`), in obnovljeno ob

vrnitvi v ospredje. S tem se zmanjša obremenitev API-jev in varčuje z baterijo na mobilnih napravah.

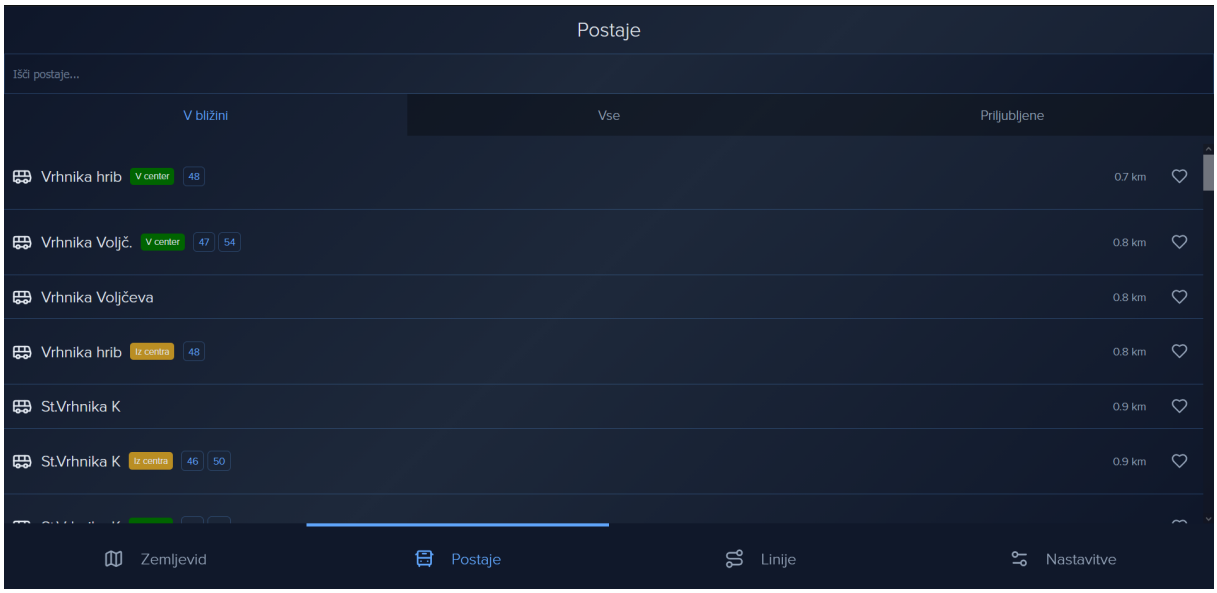
6 Uporabniški vmesnik

6.1 Zavihek Postaje

Zavihek `StationsTab` omogoča iskanje postajališč po imenu in prikaz najbližjih glede na lokacijo uporabnika. Implementiran je v treh pogledih: *V bližini* (privzeto), *Vsa postajališča* in *Priljubljena*. Primer seznama postajališč je prikazan na sliki 4.

Za zmogljivo prikazovanje dolgih seznamov, ki so pogosto dolgi več tisoč vnosov, je uporabljena tehnika virtualiziranega izrisovanja prek knjižnice `react-window`. Komponenta `VariableSizeList` izrisuje samo tiste vrstice, ki so dejansko vidne v pogledu (viewport), kar drastično zmanjša število DOM (Document Object Model) elementov in izboljša zmogljivost.

Priljubljena postajališča so shranjena v `localStorage` in so dostopna tudi brez omrežne povezave. Razdalja od uporabnika je izračunana z Haversine formulo in prikazana poleg vsakega postajališča.



Slika 4: Seznam postaj

6.2 Zavihek Linije

Zavihek `LinesTab` prikazuje prihode za izbrano postajališče, ločeno po virih (LPP, IJPP, SŽ), ter seznam vseh aktivnih linij, pridobljenih iz GPS podatkov. Omogoča iskanje linij po imenu in označevanje priljubljenih linij.

Ob kliku na prihod ali linijo se izvede funkcija `getTripFromId`, ki pridobi podrobnosti o vožnji in jih posreduje zemljevidu za prikaz trase. Prihodi so prikazani z ETA vrednostjo in absolutnim časom prihoda v formatu HH:mm, kot je prikazano na sliki 5.

Slika 5: Prikaz prihodov linij na postajo

6.3 Zavihek Nastavitve

Nastavitve omogočajo vklapljanje in izklapljanje posameznih slojev karte (avtobusi, avtobusna postajališča, vlaki, železniška postajališča) ter filtriranje prikazanih avtobusnih prevoznikov (Arriva, LPP, Nomago, Marprom, Prevozi Murska Sobota). Vse nastavitve se samodejno shranjujejo v `localStorage`.

7 Izzivi med razvojem in rešitve

7.1 Neujemanje postajališč med podsistemi

Eden najpomembnejših tehničnih izzivov je bilo dejstvo, da LPP in BrezAvta API-ja za identifikacijo postajališč ne uporabljata istih identifikatorjev. LPP ima lasten sistem šifer postajališč (`ref_id`, `station_code`), BrezAvta pa operira z identifikatorji, ki so skladni z GTFS standardom (`gtfs_id`).

Izziv: postajališče Trzin - Zalog je v bazi LPP označeno z `ref_id = 6201045`, v IJPP sistemu pa z `gtfs_id = "lpp:6201045"`.

Rešitev: za vsako postajališče se vzdržuje objekt z več identifikatorji:

```
return {
  ...stop,
  name: stop.name ?? stop.stop_name ?? "",
  gpsLocation: gpsLocation,
  ref_id: refId,        // LPP identifikator
  ijpp_id: ijppId,      // IJPP identifikator
  gtfs_id: stop.gtfs_id, // GTFS identifikator
  stopId: stop.stopId,  // SZ identifikator
  routes_on_stop: routesOnStop, // seznam lpp linij, ki ustavljajo na
    postajali u
};
```

Izsek kode 4: Interni objekt postajališča z identifikatorji vseh sistemov

Ob pridobivanju prihodov aplikacija preveri, kateri identifikator je na voljo za aktivno postajališče, in pokliče ustrezen API:

```
// LPP prihodi
const lppCode = activeStation?.ref_id || activeStation?.station_code;
if (!lppCode) { setLppArrivals([]); return; }

// IJPP prihodi
const ijppId = activeStation?.gtfs_id;
if (!ijppId) { setIjppArrivals([]); return; }

// SZ prihodi
const szId = activeStation?.stopId;
// fetchSzArrivals vrne [] ce szId ni definiran
```

Izsek kode 5: Pogojno klicanje API-jev glede na razpoložljive identifikatorje

Osnova za ta mehanizem je enotna datoteka `unified_stops_with_gtfs.json`, ki je naložena v repozitoriju `jakecernet/ijpp-json` na platformi GitHub. Ta datoteka je bila ročno sestavljena in (delno) prečiščena ter vsebuje preslikavo med identifikatorji vseh sistemov za vsako postajališče.

7.2 Heterogenost podatkovnih vmesnikov

Vsak od treh prometnih sistemov vrača podatke prek vmesnikov s povsem različnimi strukturami, avtentikacijo in obnašanjem.

LPP API zahteva, da ga naslavljamo prek Vercel proxy-ja, saj neposredni dostop iz brskalnika ni dovoljen. Podatke vrača v obliki {`data: {arrivals: [...]}`}, kjer so prihodi urejeni po `eta_min`. API je razmeroma zanesljiv in dokumentiran.

IJPP API prek strežnika `api.beta.brezavta.si` vrača prihode za postajališče s parametrom `?current=true` v obliki {`arrivals: [...]`}, kjer so časi prihoda in odhoda v sekundah od polnoči.

SŽ API prek strežnika `mapper-motis.ojpp-gateway.derp.si` temelji na odprtokodnem sistemu MOTIS [10] za načrtovanje potovanj. Za prihode vrača strukturo {`stopTimes: [...]`} z gnezdenim objektom `place`, za geometrijo poti pa kodirani polyline z natančnostjo 6 decimalnih mest (namesto standardnih 5).

Rešitev je bila implementacija ločenih funkcij, ki formatirajo izhod v enotno interno obliko.

Vse tri funkcije (`fetchLppArrivals`, `fetchIjppArrivals`, `fetchSzArrivals`) ob izhodu vrnejo tabelo objektov z istim naborom lastnosti: `etaMinutes`, `arrivalTime`, `tripName`, `tripId`, `realTime` in drugi.

7.3 Dinamično posodabljanje URL-ja za pridobivanje položajev vlakov

SŽ API za pridobivanje aktivnih vlakov zahteva časovno okno v parametrih URL-ja. Naslov vsebuje parametra `startTime` in `endTime` v formatu ISO 8601, ki morata biti ažurna ob vsakem klicu. Statičnega URL-ja, ki bi ga zgolj poklical vsakih n sekund, ni mogoče definirati.

Izziv: standardna Fetch API v JavaScriptu pričakuje statičen niz URL-ja. Definicija konstantne spremenljivke za URL vlakov bi vedno vsebovala čas ob zagonu aplikacije, ne pa čas ob

dejanskem klicu.

Rešitev: implementacija funkcije `buildSzLocationsLink()`, ki ob vsakem klicu dinamično zgradi URL z aktualnim časovnim oknom:

```
function buildSzLocationsLink() {
  const now = new Date();
  const later = new Date(now.getTime() + 60000); // +60 sekund
  return `https://mapper-motis.ojpp-gateway.derp.si/api/v1/map/trips` +
    `?min=49.41...&max=36.38...` +
    `&startTime=${encodeURIComponent(now.toISOString())}` +
    `&endTime=${encodeURIComponent(later.toISOString())}` +
    `&zoom=20`;
}
```

Izsek kode 6: Dinamična gradnja URL-ja za SŽ API

Namesto shranjene konstante se pri vsakem klicu `fetchTrainPositions()` pokliče `buildSzLocationsLink()`, ki vrne URL z vrednostmi, ki so glede na trenutni čas pravilne.

Funkcija `encodeURIComponent` poskrbi za pravilno kodiranje znakov, ki bi sicer povzročili napačno razčlenjevanje URL-ja (npr. dvopičje in plus v ISO 8601 formatu).

Dodatna zahteva API-ja je, da vsebuje bounding box (geografski pravokotnik) celotnega ozemlja, pokritega s slovenskim železniškim omrežjem. Ker SŽ API ponuja tudi lokacije vlakov za druge države, je v funkciji `fetchTrainPositions` implementiran filter po barvni kodi linije (`routeColor === "29ace2"`), ki ustreza barvni identiteti Slovenskih železnic.

7.4 Dekodiranje poti vlakov

SŽ API vrača geometrijo poti v kodirani obliki Polyline, ki je algoritem za kompresijo niza geografskih koordinat v ASCII niz [1]. Standardna implementacija algoritma predvideva natančnost 5 decimalnih mest (faktor 1e5), SŽ API pa pri nekaterih poteh uporablja natančnost 6 decimalnih mest (faktor 1e6).

Rešitev: funkcija `decodePolylineToPoints` najprej dekodira niz z nastavljenim faktorjem. Če koordinate prve točke padejo izven veljavnega geografskega območja, samodejno preskusi z faktorjem 1e6:

```
export function decodePolylineToPoints(str, precision) {
  if (!str || typeof str !== "string") return [];
  let pts = decodePolylineOnce(str, precision);
  const first = pts[0];
  // Ce tocka ni v veljavnem obsegu, poskusi z vecjo natancnostjo
  if (!first || !isValidCoord(first)) {
    const alt = decodePolylineOnce(str, precision + 1);
    if (alt[0] && isValidCoord(alt[0])) pts = alt;
  }
  return pts.filter(isValidCoord);
}
```

Izsek kode 7: Avto-zaznavno dekodiranje Polyline z variabilno natančnostjo

8 Namestitev in gostovanje

8.1 Orodje za gradnjo Vite

Gradnja aplikacije poteka z orodjem Vite [11], ki je sodobna alternativa klasičnim orodjem, kot sta Webpack in Parcel. Vite med razvojem izkorišča nativne ES module brskalnika za hitrejše posodabljanje brez ponovne gradnje celotne aplikacije. Za produkcijo Vite z orodjem Rollup "pakira" vso kodo in vire v optimizirane datoteke.

8.2 Progresivna spletna aplikacija

Aplikacija je konfigurirana kot PWA prek vtičnika `vite-plugin-pwa`. PWA je spletna aplikacija, ki jo je mogoče namestiti na domačo stran telefona ali namizja in deluje podobno kot nativna aplikacija [7]. Konfigurirani so: servisni delavec (service worker) za predpomnjenje datotek, manifest z ikonami aplikacije v ločljivostih 192×192 in 512×512 pikslov ter meta oznake za podporo na napravah Apple iOS (`apple-mobile-web-app-capable`).

8.3 Nalaganje aplikacije na GitHub Pages in Vercel

Nameščanje aplikacije je avtomatizirano s skriptami v `package.json`: ukaz `npm run build` zgradi optimizirano različico, ukaz `npm run deploy` pa jo objavi na GitHub Pages prek paketa `gh-pages`. Vercel brezstrežniške funkcije se samodejno objavijo ob vsaki potisni objavi (push) v repozitorij.

9 Zaključek

Seminarska naloga predstavlja razvoj aplikacije IJPP Tracker, ki uspešno združuje podatke treh ločenih sistemov javnega potniškega prometa v Republiki Sloveniji v enoten spletni vmesnik s sprotnim sledenjem vozil. Med razvojem je bilo ugotovljeno, da dostopnost in stabilnost javnih API-jev ostajata izziv: API IJPP vzdržuje ekipa prostovoljcev, prav tako API SŽ, kar pomeni, da lahko pride do občasnih nedostopnosti ali sprememb v strukturi podatkov, ki zahtevajo prilagoditve v aplikaciji.

10 Priloge

10.1 Izvorna koda aplikacije

Celotna izvorna koda aplikacije IJPP Tracker je dostopna v javnem repozitoriju na platformi GitHub:

- **Odjemalska aplikacija (React):** https://github.com/jakecernet/ijpp-tracker
- **Serverless funkcije (Vercel):** Vključene v istem repozitoriju v mapi `api/`
- **Seznami postajališč:** https://github.com/jakecernet/ijpp-json
- **Spletna stran:** https://tracker.cernetic.cc

11 Viri in literatura

[1] Google LLC, *Encoded Polyline Algorithm Format*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://developers.google.com/maps/documentation/utilities/polylinealgorithm

[2] Google LLC, *General Transit Feed Specification Reference*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://gtfs.org/documentation/schedule/reference/

[3] Javno podjetje Ljubljanski potniški promet d.o.o., *LPP Ljubljana - uradni API*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://data.lpp.si/doc/

[4] MapLibre Contributors, *MapLibre GL JS*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://maplibre.org/maplibre-gl-js/docs/

[5] Meta Open Source, *React – A JavaScript library for building user interfaces*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://react.dev/

[6] Mozilla Foundation, *Cross-Origin Resource Sharing (CORS)*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

[7] Mozilla Foundation, *Progressive Web Apps*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

[8] Nedžad Beus, *Slovenske železnice API*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://map.vlak.si/

[9] OJPP, *BrezAvta*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://beta.brezavta.si/

[10] F. Pfeiffer in S. Lenz, *MOTIS: Multi-modal Travel Information System*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://github.com/motis-project/motis

[11] Vite Contributors, *Vite – Next Generation Frontend Tooling*, dostopano: 1. 4. 2026, 2026. spletni naslov: https://vite.dev/

Izjava o avtorstvu

Izjavljam, da je seminarska naloga z naslovom *Aplikacija za sledenje javnemu prometu* v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

14. april 2026 Jaka Černetič